

Travel Companion: Scaling to 100,000 Concurrent Users

Architecture & Infrastructure Strategy | Team 11 | Stack: ElysiaJS (Bun) · Neon PostgreSQL · React Native (Expo) | Cloud: AWS + Cloudflare

1. Overview

This document outlines the infrastructure and engineering strategy to serve 100,000 simultaneous users on the Travel Companion application. The backend sustains continuous load across authentication, destination/place discovery, itinerary management, Google Maps API proxying, real-time chat, sync, and social features. The offline mode is limited to active trips without connectivity. The architecture is built on AWS for cloud hosting and Cloudflare for CDN and edge delivery, designed to scale horizontally under peak demand.

2. Backend Deployment on AWS

2.1 Kubernetes with Horizontal Auto-Scaling (EKS)

The ElysiaJS/Bun Docker image is deployed to Amazon EKS (Elastic Kubernetes Service). Kubernetes Horizontal Pod Autoscaler (HPA) scales pods based on CPU and memory thresholds - scaling out at 60% CPU utilization. A minimum of 3 replicas always run across multiple AWS Availability Zones, **scaling to 30+ pods** under peak load. Bun's efficient async I/O allows a pod to handle 300–500 concurrent HTTP and WebSocket connections, so 30 pods comfortably support 100k users.

- **EKS Fargate** removes the need to manage EC2 node groups directly. Pods are provisioned on demand, billed per vCPU/memory second - cost-efficient for variable load.
- **Cluster Autoscaler** runs alongside HPA so underlying EC2 capacity grows with pod demand and shrinks during off-peak hours.
- **AWS Application Load Balancer (ALB)** sits in front of the cluster. Distributes incoming application traffic across multiple pods.

2.2 Multi-Region Deployment

We deploy clusters in three AWS regions: us-east-1 (primary), eu-west-1, and ap-southeast-1. **AWS Route 53** with latency-based routing directs each user to the nearest healthy region. An active/passive configuration covers v1: the primary region handles all traffic, and Route 53 health checks trigger automatic DNS failover to the secondary region within 60 seconds of a primary failure. For a travel app with globally distributed users, multi-region routing materially reduces API latency - a user in Europe should not be hitting Virginia.

3. CDN & Static Asset Delivery (Cloudflare)

City packs - bundled Destination, Place, Phrase data, and cached map tiles - are the heaviest client downloads. These are write-once, read-many assets. We store city packs on **Cloudflare R2** (S3-compatible object storage) and serve them through Cloudflare's CDN:

- **Zero egress fees on R2** make it significantly cheaper than S3 + CloudFront for large binary downloads. Packs are versioned via the packVersion integer; Cloudflare caches each version with immutable Cache-Control headers.
- **The Qwen 3.5 2B GGUF model file** is a one-time download per device, served from EXPO_PUBLIC_MODEL_CDN_URL backed by Cloudflare R2. Cloudflare's global edge network ensures fast downloads in Asia, Europe, and the Americas.
- **Cloudflare Workers** can handle edge-level rate limiting and bot mitigation before traffic ever reaches AWS, reducing origin load.

Estimated CDN cost: at 100k users each downloading one ~200 MB city pack, that is ~20 TB egress. On Cloudflare R2, egress to the internet is free. Storage at \$0.015/GB/month for 1 TB of pack data = ~\$15/month.

4. Caching Strategy

4.1 Client-Side (TanStack Query)

TanStack Query (eden-tanstack-react-query) is configured with **staleTime** and **gcTime** tuned to each data type's churn rate:

- Destinations, Places, and Phrases: staleTime of 12 hours. Team-seeded data changes rarely.
- Nearby Travelers: staleTime of 30 seconds. Social presence requires freshness.
- User Trips and ItineraryItems: staleTime of 5 minutes with optimistic updates on mutation. The offline outbox manages conflict resolution on reconnect.
- Real-time chat messages: TanStack Query cache is bypassed for the active chat WebSocket stream; historical messages are cached and paginated via cursor-based queries.

4.2 Server-Side (Upstash Redis)

Upstash Redis provides serverless, per-request-billed Redis - no idle costs, scales automatically with request volume. We use it for:

- **JWT blacklist:** O(1) lookup for invalidated refresh tokens, with TTL (time to live) matching the token expiry window.
- **Rate limit counters:** Sliding window counters per user and IP (see Section 8).
- **Destination and pack data:** Destination lists and packVersion values cached with a 12-hour TTL; invalidated on admin update.
- **Nearby Travelers:** Opt-in user list per destination cached for 60 seconds, preventing repeated PostgreSQL queries on social polling.
- **Google Maps API responses:** Place details and geocoding results cached by query hash for 24 hours (see Section 7).

Upstash pricing: ~\$0.20 per 100k commands. At 100k active users generating ~50 cache reads per session, expect \$10–\$100/day depending on session depth.

5. Database Scaling (Neon PostgreSQL)

5.1 Neon Serverless with Connection Pooling

We use Neon as our managed PostgreSQL provider. Neon's serverless compute scales to zero on idle and scales up on demand, eliminating over-provisioning costs. Neon's built-in **PgBouncer** (transaction mode) sits between EKS pods and the database, capping total Postgres connections regardless of pod count. ElysiaJS/Bun maintains a process-level connection pool per pod; the Neon pooler aggregates these into a manageable connection ceiling.

- Neon replicates data across AWS availability zones with automatic failover and point-in-time recovery (PITR) - no additional backup configuration required.
- Database migrations use the expand/contract pattern: schema changes are backward-compatible with the running app version.

5.2 Read Replicas

We add one Neon **read replica** directed at read-heavy, low-churn queries: Destination/Place/Phrase lookups, social connection lists, and analytics. All writes go to the primary. This halves read pressure on the primary at 100k users.

5.3 Indexing & Query Optimization

Indexes are defined on: `userId` (all user-scoped tables), `destinationId` (Trips, Places, Phrases), `status` (Connections), and composite indexes on commonly joined columns.

6. Real-Time Chat & Concurrency

6.1 WebSocket Architecture for Chat

Real-time text chat runs over persistent WebSocket connections. At 100k concurrent users, a single pod cannot hold all connections - pod restarts would drop every connection on that pod, and connections are distributed across many pods. We use Upstash **Redis Pub/Sub** as a message bus to bridge them: when User A (on Pod 1) sends a message to User B (on Pod 7), Pod 1 publishes to a Redis channel and Pod 7 picks it up and delivers it to User B's WebSocket. This decouples message delivery from which pod a user happens to be connected to.

- **Connection limits:** Each pod handles up to 500 WebSocket connections. At 30 pods, that supports 15,000 simultaneous chat connections - sufficient for the active-chat fraction of 100k users.

6.2 Offline Sync Concurrency

When a user reconnects after an offline trip, the outbox replays queued writes in order - adds, edits, deletes, reorders, and toggles. Conflicts are resolved by a fixed rule: server wins on deletes; client wins on edits. Each outbox entry carries a `clientGeneratedId` to prevent duplicate rows if the same entry is replayed more than once.

6.3 Push Notifications

Push notifications via the Expo Push Service are fire-and-forget. We batch sends using Expo's `/push/send` endpoint (up to 100 receipts per request) to stay within rate limits and minimize outbound HTTP connections from the backend.

7. External API Management

7.1 Google Maps API

All Google Maps API calls (Places, Geocoding, Directions, Distance Matrix) are proxied through the backend - the API key is never exposed to the client. Server-side Redis caching is applied to every Maps response:

- Place details and search results: cached by query hash for 24 hours. At 100k users querying the same destinations, the cache absorbs most upstream calls.
- Geocoding results: cached indefinitely (coordinates for a place name do not change). Stored in Redis with no TTL.
- Google Maps Platform pricing is per-request. Without caching, 100k users performing 10 place lookups each = 1M API calls/day at ~\$3.50/1k = \$3,500/day. With a 95% cache hit rate, actual upstream calls drop to ~50k/day = \$175/day. Caching is the primary cost control mechanism for Maps.

7.2 Other Third-Party APIs

The same proxy-and-cache pattern applies to any additional external APIs (weather, currency exchange, etc.): all calls go through the backend, responses are cached in Redis by query parameters, and per-minute upstream counters prevent quota exhaustion. If an upstream API fails, the backend returns the last cached value with a staleness indicator rather than propagating an error to the client.

8. Rate Limiting

Rate limiting is enforced at two layers to protect both the backend and upstream API quotas:

- **Cloudflare (edge layer):** Managed rate limiting rules block abusive IPs before traffic reaches AWS. Auth endpoints (`/auth/login`, `/auth/register`) are limited to 10 requests/minute per IP. DDoS protection is handled by Cloudflare's network-level mitigation automatically.
- **Upstash Redis (application layer):** ElysiaJS middleware applies a sliding window counter (`rate:userId:window`) on every request. Limits: 120 requests/minute for general API endpoints, 10/minute for auth, 20/minute for sync, 60/minute for chat message sends. Requests exceeding limits receive a 429 with a `Retry-After` header.
- **Upstream API quota guards:** Per-minute counters for Google Maps API calls gate requests before they reach Google. If the per-minute budget is exhausted, the backend serves the cached response or returns a 503 with a clear error code.

9. CI/CD Pipeline (GitHub Actions)

Our GitHub Actions pipeline handles the full deployment lifecycle on every merge to main (for backend):

- **Test:** Run backend unit and integration tests. Enforce >70% coverage gate. Fail fast on any test regression.
- **Build & Push:** Build the ElysiaJS/Bun Docker image and push to Amazon ECR (Elastic Container Registry). Images are tagged with the commit SHA for traceability.
- **Deploy:** Apply Kubernetes manifests via kubectl rollout or Helm upgrade. EKS performs a rolling deployment - new pods pass readiness and liveness probes before old pods terminate, guaranteeing zero downtime.
- **Verify:** Post-deploy smoke tests hit the staging and production health endpoints. A failed smoke test triggers an automatic rollback to the previous image tag via kubectl rollout undo.

10. Logging, Metrics & Error Tracking

Concern	Tool	Notes
Structured Logging	Axiom	JSON logs from Bun/ElysiaJS shipped to Axiom. Free tier: 1 GB/day ingest. Queryable log explorer for prod debugging.
Metrics & Dashboards	Grafana Cloud + Prometheus	Scrape /metrics from ElysiaJS pods. Grafana Cloud free tier: up to 10k active series. Custom dashboards for RPS, latency, WebSocket connections.
Error Tracking	Sentry	SDK in ElysiaJS (backend) and React Native (mobile). Captures stack traces, release versions, and user context. Team plan: \$26/month.
Uptime Monitoring	Better Stack Uptime	Synthetic checks every 30s on health endpoints and WebSocket handshake. PagerDuty or Slack alerts. ~\$29/month.
Product Analytics	PostHog	\$0 on cloud up to 1M events/month. Feature flags for staged rollouts.

11. Redundancy & Resilience

- **Multi-Zone database:** Neon replicates across AWS availability zones with automatic failover. Recovery Time Objective (RTO) is under 30 seconds.
- **Multi-region failover:** Route 53 health checks on the primary region trigger DNS failover to eu-west-1 within 60 seconds of a primary failure. The secondary region runs a warm standby EKS cluster pointing to a Neon read replica promoted to primary on failover.
- **Backups:** Neon covers up to 7 days of point-in-time recovery. City pack files on Cloudflare R2 are versioned; prior versions are retained for 30 days. Restore procedures are tested in staging monthly.

12. High-Level Cost Analysis at 100k Concurrent Users

Estimates assume sustained 100k concurrent users. Serverless and autoscaling components (EKS Fargate, Neon, Upstash) scale down significantly during off-peak hours, making average monthly costs lower than peak figures.

Component	Service	Est. Monthly Cost
Kubernetes Cluster (3–30 pods)	AWS EKS + Fargate	\$400 – \$1,000
Load Balancer	AWS ALB	\$25 – \$60
Database (primary + 1 replica)	Neon Scale Plan	\$200 – \$500
Redis / Cache / Pub-Sub	Upstash Redis	\$50 – \$200
CDN & Object Storage	Cloudflare R2 + CDN	\$15 – \$80
Container Registry	Amazon ECR	\$5 – \$20
Error Tracking	Sentry Team	\$26
Logging	Axiom	\$0 – \$100
Metrics & Tracing	Grafana Cloud	\$0 – \$50
Uptime Monitoring	Better Stack Uptime	\$29
DNS & Failover	AWS Route 53	\$5 – \$15
Google Maps API (post-cache)	Google Maps Platform	\$150 – \$500
Secrets Management	AWS Secrets Manager	\$10 – \$20

Estimated total: \$915 – \$2,596 per month at peak. The largest variable cost is Google Maps API - aggressive Redis caching (targeting >95% hit rate) is the single most impactful cost optimization. Expo push notification costs are usage-based (\$0.01/1k after the free tier) and are not included above.

13. Load Testing

Load testing against a production-scale Neon dataset runs in staging before every major release. We use **k6** to simulate 100k concurrent users exercising the following scenarios simultaneously: authentication, itinerary read/write, Google Maps proxy, WebSocket chat connection, and Nearby Travelers polling. Database connection exhaustion is the most common failure mode in Bun/Node apps before CPU is ever the bottleneck - connection pool sizing is validated first in every load test run. Any endpoint exceeding a 500ms p99 latency threshold at 100k simulated users is treated as a blocking issue before launch.